

提出 TIME-LLM，一个重编程框架，用于适配大型语言模型进行时间序列预测，同时保持骨干模型不变。核心思想是将输入时间序列重编程为更自然适合语言模型能力的文本原型表示。为了进一步增强模型对时间序列概念的推理能力，我们引入了“提示即前缀”（PaP），这是一种新颖的思路，通过自然语言模态的附加上下文和任务指令来丰富输入时间序列，从而为应用于重编程输入的所需转换提供声明性指导。语言模型的输出随后被投影以生成时间序列预测。

第一张图：

重编程大语言模型（LLM）的示意图，对比了（a）面向特定任务的学习与（b）模型微调。

（a）是为特定任务和领域（如交通预测）量身定制的，并在小规模数据上进行端到端训练。尽管在狭义任务上表现良好，但这些模型缺乏对多样化时间序列数据的通用性和泛化能力。

（b）模型微调：

预训练模型的有效性可以通过微调适应各种下游任务，而无需从头进行昂贵的训练（受这些成功的启发，近期研究聚焦于时间序列预训练模型（TSPTMs）的开发。其中第一步涉及使用不同策略进行时间序列预训练，如监督式（或自监督学习）。这使得模型能够学习表示各种输入时间序列。预训练后，可以在相似领域进行微调以执行特定任务（示例如图 1(b)所示。TSPTM 的发展借鉴了 NLP 和 CV 中预训练和微调的成功，但由于数据稀疏性，仍然局限于较小规模。

模型架构相关：

我们的方法包含三个主要组件：（1）输入变换，（2）预训练且冻结的 LLM，以及（3）输出投影。首先，将多变量时间序列划分为 N 个单变量时间序列，随后独立处理（Nie等，2023）。第 i 个序列记为 $\mathbf{X}^{(i)} \in \mathbb{R}^{1 \times T}$ ，它经过归一化、分块和嵌入，然后与学习到的文本原型进行重编程以对齐源模态和目标模态。然后，我们通过提示和重编程的补丁一起增强 LLM 的时间序列推理能力，生成输出表示，这些表示被投影到最终的预测结果 $\hat{\mathbf{Y}}^{(i)} \in \mathbb{R}^{1 \times H}$ 。

考虑以下问题：给定一个历史观测序列 $\mathbf{X} \in \mathbb{R}^{N \times T}$ ，包含 N 个不同的一维变量，跨 T 个时间步，我们的目标是重编程一个大型语言模型 $f(\cdot)$ 以理解输入时间序列，并准确预测未来 H 个时间步的读数 $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times H}$ ，总体目标是最小化真实值 $\mathbf{Y}$ 与预测值之间的均方误差，即 $\frac{1}{H} \sum_{h=1}^H \|\hat{\mathbf{Y}}_h - \mathbf{Y}_h\|_F^2$ 。

输入嵌入。每个输入通道 $\mathbf{X}^{(i)}$ 首先通过可逆实例归一化（RevIN）分别归一化为零均值和单位标准差，以缓解时间序列分布偏移（Kim等，2021）。然后，我们将 $\mathbf{X}^{(i)}$ 划分为若干个连续的重叠或非重叠补丁

（Nie等，2023），长度为 L_p ；因此输入补丁总数为 $P = \left\lfloor \frac{(T-L_p)}{S} \right\rfloor + 2$ ，其中 S 表示水平滑动步长。

其潜在动机有两方面：（1）通过将局部信息聚合到每个补丁中更好地保留局部语义信息；（2）作为 token 化，形成紧凑的输入 token 序列，减少计算负担。给定这些补丁 $\mathbf{X}_P^{(i)} \in \mathbb{R}^{P \times L_p}$ ，我们将其嵌入为 $\hat{\mathbf{X}}_P^{(i)} \in \mathbb{R}^{P \times d_m}$ ，采用简单的线性层作为补丁嵌入器，以创建维度 d_m 。

Patch Reprogramming:

为弥合这一差距，我们提出使用骨干模型中预训练的词嵌入 $\mathbf{E} \in \mathbb{R}^{V \times D}$ 来重编程 $\hat{\mathbf{X}}_P^{(i)}$ ，其中 V 是词汇表大小。然而，没有先验知识表明哪些源token直接相关。因此，简单地利用 $\mathbf{E}$ 将导致庞大且可能密集的重编程空间。一个简单的解决方案是通过线性探测 $\mathbf{E}$ 来维护一个小型文本原型集合，记为 $\mathbf{E}' \in \mathbb{R}^{V' \times D}$ ，其中 $V' \ll V$ 。如图3(a)所示。文本原型学习连接语言线索，例如“short up”（红线）和“steady down”（蓝线），然后组合起来表示局部补丁信息（例如，用“short up then down steadily”来描述补丁5），而不会离开语言模型预训练的空间。这种方法高效，并允许自适应选择相关的源信息。为实现这一点，我们采用多头交叉注意力层。具体地，对于每个头 $k = \{1, \dots, K\}$ ，我们定义查询矩阵 $\mathbf{Q}_k^{(i)} = \hat{\mathbf{X}}_P^{(i)} \mathbf{W}_k^Q$ ，键矩阵 $\mathbf{K}_k^{(i)} = \mathbf{E}' \mathbf{W}_k^K$ ，值矩阵 $\mathbf{V}_k^{(i)} = \mathbf{E}' \mathbf{W}_k^V$ ，其中 $\mathbf{W}_k^Q \in \mathbb{R}^{d_m \times d}$ ， $\mathbf{W}_k^K, \mathbf{W}_k^V \in \mathbb{R}^{D \times d}$ 。这里 D 是骨干模型的隐藏维度， $d = \lfloor \frac{d_m}{K} \rfloor$ 。然后，每个注意力头中重编程时间序列补丁的操作定义为：

$$\mathbf{Z}_k^{(i)} = \text{ATTENTION}(\mathbf{Q}_k^{(i)}, \mathbf{K}_k^{(i)}, \mathbf{V}_k^{(i)}) = \text{SOFTMAX}\left(\frac{\mathbf{Q}_k^{(i)} \mathbf{K}_k^{(i)\top}}{\sqrt{d_k}}\right) \mathbf{V}_k^{(i)}. \quad (1)$$

通过聚合每个头中的 $\mathbf{Z}_k^{(i)} \in \mathbb{R}^{P \times d}$ ，我们得到 $\mathbf{Z}^{(i)} \in \mathbb{R}^{P \times d_m}$ 。然后将其线性投影以对齐骨干模型的隐藏维度，得到 $\mathbf{O}^{(i)} \in \mathbb{R}^{P \times D}$ 。

d_k 就是 d

k 个头上第 i 个补丁

键矩阵代表：每一个 **Prototype** 是什么。

值矩阵保存的是 **Prototype** 真正携带的信息。

公式中 QK 的转置表示每个 Patch 与每个 Prototype 有多接近。

第二步

除以

$$\sqrt{d_k}$$

这是Transformer标准缩放。

原因：

防止内积过大，

Softmax饱和。

第三步

Softmax

得到

Patch1

↓

0.61

0.08

0.20

...

这些就是

Attention Weight。

意思：

Patch更像哪些Prototype。

第四步

乘

Value

$$AV$$

即

0.61 Prototype1

+

0.08 Prototype2

+

0.20 Prototype3

最后得到

新的Patch表示:

$$Z_k^{(i)}$$

即

Reprogrammed Patch。

K 个 head 学习不同的语义

采用多头交叉注意力 (Multi-head Cross Attention) 机制, 以时间序列 Patch 作为 Query、文本原型作为 Key 和 Value, 计算每个 Patch 与不同文本原型之间的相关性。最终, 每个时间序列 Patch 都被表示为多个文本原型的加权组合, 从而生成位于 LLM 词嵌入空间中的重编程表示 (Reprogrammed Patch Embedding)。

提示即前缀：

图4：提示示例。< > 和 < > 是任务特定配置和计算得到的输入统计量。

两种提示方式的示意图如图3(b)所示。在“补丁即前缀”中，语言模型被提示以自然语言表述预测时间序列的后续值。这种方法遇到了一些限制：（1）语言模型在处理高精度数字时通常敏感性较低，没有外部工具的帮助，因此在准确处理长期预测任务时面临重大挑战；（2）不同语言模型需要复杂的定制后处理，因为它们是在不同的语料库上预训练的，并且在生成高精度数字时可能采用不同的token化类型，导致预测结果以不同的自然语言格式表示，例如 ['0', '1', '6', '1'] 和 ['0', '1', '61'] 来表示小数0.61。

相比之下，提示即前缀巧妙地避免了这些限制。在实践中，我们确定了构建有效提示的三个关键组成部分：（1）数据集上下文，（2）任务指令，和（3）输入统计量。提示示例如图4所示。数据集上下文为LLM提供了有关输入时间序列的基本背景信息，这些信息在不同领域通常表现出不同的特征。任务指令是引导LLM将补丁嵌入转换为特定任务的关键指导。我们还用重要的统计量（如趋势和滞后）丰富输入时间序列，以促进模式识别和推理。

短期预测评估函数：
三个都是越小越好

指标	衡量内容	是否归一化	是否与基线比较	越大越好?	
SMAPE	百分比误差	是（按真实值与预测值之和）	否	否，越小越好	
MASE	相对误差	是（按季节性 Naive 误差缩放）	是	否，越小越好	
OWA	综合指标	是	是（相对于 Naive2）	否，越小越好	

SMAPE:

普通的 MAPE 定义为:

$$\text{MAPE} = \frac{100}{H} \sum_{h=1}^H \frac{|Y_h - \hat{Y}_h|}{|Y_h|}$$

它的问题是:

当

$$Y_h \rightarrow 0$$

时,

分母趋近于0,

误差会无限放大。

因此不同数据集之间很难公平比较。

SMAPE 将分母改为

$$|Y_h| + |\hat{Y}_h|$$

即同时考虑真实值和预测值, 因此称为 **Symmetric (对称)**。

2. MASE 的基本思想

MASE 的核心思想非常简单:

不要直接看模型误差有多大, 而是看它相对于一个简单基线模型 (Naive Forecast) 的误差有多大。

换句话说, MASE 衡量的是:

模型的平均绝对误差, 是基线预测误差的多少倍。

因此, 它是一个**相对误差指标 (Relative Error)**。

(1) 分子：模型预测误差

$$\frac{1}{H} \sum_{h=1}^H |Y_h - \hat{Y}_h|$$

即模型预测结果与真实值之间的平均绝对误差（MAE）。

它表示：

模型到底预测错了多少。

(2) 分母：Naive Forecast 的误差

$$\frac{1}{H-s} \sum_{j=s+1}^H |Y_j - Y_{j-s}|$$

其中：

- s ：季节周期（Seasonality）
 - 小时数据： $s = 24$
 - 日数据： $s = 7$
 - 月数据： $s = 12$
- Y_{j-s} ：前一个周期对应位置的真实值。

这一项实际上计算的是：

如果直接用“上一周期的数据”作为预测值，会产生多大的平均误差。

因此，分母代表的是季节性 Naive Forecast 的误差。

3. OWA (Overall Weighted Average)

论文定义：

$$OWA = \frac{1}{2} \left(\frac{SMAPE}{SMAPE_{Naive2}} + \frac{MASE}{MASE_{Naive2}} \right)$$

 Time

(1) Naive2是什么？

Naive2

是

M4 Competition

官方Baseline。

即：

Seasonal Naive Forecast。

所有参赛算法

都要与它比较。